

[CS230] Improving Deep Neural Networks: Data Augmentation & Early Stopping

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-4. Neural Networks Basics - *Completed*

Chapter 5. Practical Aspects of Deep Learning (Current Unit)

- 5.1-5.3 Regularization (L2, Dropout) - *Completed*
- **5.4 Data Augmentation & Early Stopping**
 - * Concept: Free Data & Time Machine
 - * Augmentation Techniques (Flip, Crop, Rotate)
 - * Early Stopping Mechanism (Patience)
 - * Implementation: On-the-fly Generation
- 5.5 Input Normalization - *Upcoming*

지난 시간 복습 및 연결

우리는 L2 정규화와 드롭아웃이라는 강력한 수학적 기법으로 과대적합(Overfitting)을 억제했습니다. 오늘은 조금 더 '실용적이고(Practical)' '경제적인(Economical)' 접근법을 다룹니다. 데이터를 더 모으는 것은 비쌉니다. 하지만 가지고 있는 데이터를 변형해서 '공짜 데이터'를 만드는 기술(Augmentation)과, 학습을 가장 좋은 타이밍에 멈추는 '타임머신' 기술(Early Stopping)은 비용 대비 효과가 엄청납니다.

1 Unit Overview

□ 핵심 목표

이 단원은 모델의 일반화 성능을 높이는 가장 직관적이고 가성비 좋은 두 가지 기법을 다룹니다.

- **Data Augmentation:** 이미지를 변형하여 데이터셋을 팽튀기하고, 모델에게 불변성(Invariance)을 가르칩니다.
- **Early Stopping:** 과대적합이 시작되기 직전에 학습을 멈추는 알고리즘을 구현합니다.
- **On-the-fly:** 디스크 용량을 아끼기 위해 학습 도중 실시간으로 데이터를 변형하는 파이프라인을 이해합니다.

2 Essential Terminology

용어	설명	비유
Data Augmentation	원본 데이터를 변형해 가짜 데이터를 생성	복사기로 문제집 복사하기 (근데 약간 변)
Early Stopping	성능 악화 시점에 학습 중단	박수 칠 때 떠나라
Patience	성능이 안 좋아져도 기다려주는 횟수	"한 번만 더 기회를 줄게"
On-the-fly	미리 저장하지 않고 필요할 때 즉석 생성	주문 들어오면 요리하기 (미리 해두면 상)

3 Core Concepts: 공짜 점심은 없다

3.1 1. Data Augmentation (데이터 증강)

모델에게 "고양이는 뒤집어도, 어두워도, 잘려도 고양이다"라는 사실을 가르칩니다.

- **Mirroring (Flipping):** 거울처럼 좌우 반전. (숫자 인식 등 방향이 중요한 데이터엔 금지!)
- **Random Cropping:** 이미지의 일부분을 무작위로 잘라냄.
- **Rotation / Shearing:** 회전 및 비틀기.
- **Color Jittering:** 밝기, 채도 등에 노이즈 추가.

□ On-the-fly Generation (실시간 생성) (실전 팁)

"교수님, 변형된 이미지를 하드디스크에 저장해두고 써야 합니까?" 절대 아닙니다. 1TB 짜리 데이터셋을 10배 증강하면 10TB가 됩니다. 감당할 수 없습니다. CPU가 학습 도중에 실시간으로 변형해서 GPU에게 넘겨주는 방식을 사용합니다. 디스크 용량은 그대로 유지 됩니다.

3.2 2. Early Stopping (조기 종료)

학습 곡선(Learning Curve)을 보다가, Train Error는 줄지만 Dev Error가 다시 올라가려는 순간 (과대적합 시작점)에 멈춥니다.

- **원리:** 학습을 오래 하면 가중치 W 가 점점 커져서 복잡한 패턴을 익히게 됩니다. Early Stopping은 W 가 너무 커지기 전에 멈추므로, 수학적으로 **L2 정규화와 유사한 효과**를 냅니다.
- **단점 (Orthogonalization):** Andrew Ng 교수는 이 방식이 'Bias 줄이기'와 'Variance 줄이기'를 동시에 건드리기 때문에(직교화 위배), 튜닝이 복잡해질 수 있다고 지적합니다. 하지만 편해서 많이 씁니다.

4 Implementation: On-the-fly Pipeline

데이터 증강은 NumPy로 간단히, 조기 종료는 클래스로 구현하여 원리를 파악합니다.

```
1 import numpy as np
2 import copy
3
4 class DataAugmentor:
```

```

5  @staticmethod
6  def random_flip(image, p=0.5):
7      """좌우 반전"""
8      if np.random.rand() < p:
9          return np.fliplr(image)
10     return image
11
12     @staticmethod
13     def random_crop(image, crop_size=(200, 200)):
14         """무작위 위치 자르기"""
15         h, w, _ = image.shape
16         top = np.random.randint(0, h - crop_size[0])
17         left = np.random.randint(0, w - crop_size[1])
18         return image[top:top+crop_size[0], left:left+crop_size[1], :]
19
20 class EarlyStopping:
21     def __init__(self, patience=5, min_delta=0.0):
22         self.patience = patience # 참을성 횟수()
23         self.min_delta = min_delta # 최소 개선폭
24         self.counter = 0
25         self.best_loss = np.inf
26         self.best_model = None
27         self.stop = False
28
29     def check(self, val_loss, model_params):
30         if val_loss < (self.best_loss - self.min_delta):
31             # 성능 개선! -> 저장 및 카운터 초기화
32             self.best_loss = val_loss
33             self.counter = 0
34             # 중요: 로deepcopy 값 자체를 복사해둬야 함 참조(복사 금지)
35             self.best_model = copy.deepcopy(model_params)
36         else:
37             # 성능 정체악화/ -> 카운터 증가
38             self.counter += 1
39             print(f"EarlyStopping counter: {self.counter}/{self.patience}")
40             if self.counter >= self.patience:
41                 self.stop = True
42
43     def restore(self):
44         return self.best_model

```

Listing 1: Data Augmentation & Early Stopping

5 FAQ & Pitfalls

Deep Copy를 안 쓰면 생기는 일 (오해 방지 가이드)

파이썬에서 'best_model = model' , 'best_model' 'model' , 'model' 'best_model' . 'copy.de

Q. Data Augmentation을 Test Set에도 적용하나요?

A. 보통은 안 합니다. 하지만 성능을 극대화하기 위해 'Test Time Augmentation (TTA)'이라는 기법을 쓰기도 합니다. 테스트 이미지를 5가지로 변형해서 예측한 뒤 평균을 내는 것입니다. (대회용 테크닉)

다음 단계 (Next Step)

이제 우리는 과대적합을 막는 모든 무기(L2, Dropout, Augmentation, Early Stopping)를 갖췄습니다. 방어 준비는 끝났습니다.

이제 공격(학습 속도)을 강화할 차례입니다. 경사 하강법(Gradient Descent)은 너무 정직해서 느립니다. 다음 시간에는 [Optimization Algorithms]으로 넘어가서, 경사 하강법에 가속도를 붙이는 Momentum, 보폭을 조절하는 Adam 등 최신 최적화 기법을 배우겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Augmentation:** 데이터를 변형해 양을 늘리고 불변성(Invariance)을 학습시킨다.
2. **On-the-fly:** 디스크 절약을 위해 학습 도중 실시간으로 변형한다.
3. **Early Stopping:** Dev Error가 오르기 시작하면 멈춘다. (과대적합 방지)
4. **Patience:** 일시적인 성능 저하를 견디기 위해 인내심(Patience) 값을 설정한다.